



CENTRO OFICIAL DE FORMACIÓN PROFESIONAL CESUR

CICLO FORMATIVO DE GRADO SUPERIOR EN

DESARROLLO DE APLICACIONES WEB

MUSARUM

Diseño e implementación de un ecosistema multiplataforma de gestión, análisis y streaming de audioteca privadas mediante enriquecimiento automatizado de metadatos y acceso vía redes personales privadas

Carlos Sánchez Miranda



ÍNDICE DE CONTENIDOS

1. INTRODUCCIÓN	5
2. ANÁLISIS INICIAL	6
2.1. Conceptos Específicos	6
2.2. Justificación del Proyecto	6
3. OBJETIVOS	8
3.1. Objetivos Funcionales	8
3.2. Objetivos No Funcionales	9
4. HISTORIAS Y CASOS DE USO	10
4.1. Actores del Sistema	10
4.2. Historias de Usuario	10
4.3. Casos de Uso	11
5. METODOLOGÍA	14
6. ARQUITECTURA DEL SISTEMA	17
6.1. Visión General	17
6.2. Diseño del Servidor	18
6.2.1. Arquitectura en Capas	18
6.2.2. Variables de Entorno y Configuración	19
6.2.3. Transferencia de Datos	20
6.3. Diseño del Cliente Web	20
6.4. Diseño de Persistencia	21
6.4.1. Diagrama Entidad – Relación	21
6.4.2. Integridad Referencial	23
6.4.3. Almacenamiento de Ficheros Físicos	23
6.5. Patrones de Diseño	24
7. DISEÑO DE INTERFAZ	25

8. IMPLEMENTACIÓN TÉCNICA	27
8.1. Stack Tecnológico	27
8.1.1. Backend	27
8.1.2. Frontend	27
8.1.3. Persistencia	27
8.1.4. Librerías Auxiliares	28
8.1.5. APIs Externas	28
8.1.6. Herramientas de Desarrollo	28
8.2. Seguridad	29
8.3. Funcionalidades Principales	29
8.3.1. Gestión de Usuarios y Autenticación	29
8.3.2. Carga y Gestión de Audioteca Musical	30
8.3.3. Streaming de Audio	30
8.3.4. Enriquecimiento de Datos	30
8.3.5. Listas de Reproducción	31
8.3.6. Letra Sincronizada	31
8.3.7. Reconocimiento de Voz	32
8.3.8. Modos de Reproducción	32
8.3.9. Estadística y Analítica	33
8.3.10. Administración del Sistema	33
8.4. Optimizaciones de Rendimiento	34
9. DESPLIEGUE Y CONFIGURACIÓN	35
10. PRUEBAS Y RESULTADOS	37
11. CONCLUSIONES Y MEJORAS	40
12. BIBLIOGRAFÍA	42

1. INTRODUCCIÓN

El modelo actual de consumo musical ha convertido al oyente en un usuario dependiente de plataformas de terceros. Los servicios de streaming comerciales concentran el control sobre las audioteca, los metadatos y el historial de escucha en servidores ajenos, exponiendo al usuario a la pérdida de contenido por decisiones de licencia, a la fragmentación entre múltiples suscripciones y a la ausencia de herramientas reales de análisis sobre sus propios hábitos. Al mismo tiempo, los reproductores multimedia locales tradicionales, aunque permiten conservar la propiedad de los archivos, ofrecen interfaces limitadas que no satisfacen las expectativas de una experiencia moderna.

Musarum nace para cerrar esa brecha. Es un ecosistema de streaming de audio diseñado bajo una arquitectura cliente-servidor desacoplada que permite al usuario gestionar su propia audioteca musical de forma privada y autónoma. El sistema integra un backend desarrollado en Spring Boot y un frontend en Angular, complementado con una aplicación de escritorio en Electron. A diferencia de los servicios comerciales masivos, este proyecto se centra en la filosofía del autoalojamiento, permitiendo la indexación inteligente de miles de pistas locales mediante el escaneo de metadatos ID3 y su enriquecimiento automático a través de APIs externas como Deezer o LRCLIB.

La motivación central de este desarrollo surge de una inquietud personal: administrar y disfrutar una gran colección musical propia a través de una interfaz rica y con funcionalidades avanzadas, superando las carencias de los reproductores locales sin renunciar a la propiedad de los archivos. El objetivo no ha sido construir un simple reproductor, sino ofrecer una experiencia de usuario a la altura de las plataformas de la industria, aplicada a un catálogo privado.

Para lograrlo, Musarum implementa una infraestructura segura mediante contenedores Docker y redes privadas virtuales, ofreciendo un ecosistema donde la integridad de la música y la privacidad del historial de escucha pertenecen exclusivamente al usuario, eliminando intermediarios y garantizando la persistencia de los datos.

2. ANÁLISIS INICIAL

2.1. Conceptos Específicos

Se muestran conceptos usados en esta memoria además de conceptos generales sobre la arquitectura de Musarum pertenecientes tanto al dominio de software como a la música y gestión multimedia:

Metadatos y Etiquetas ID3 (ID3 Tags). Etiquetas de información incrustadas dentro de los archivos de audio como: artista, álbum, año o carátula.

BPM y Tonalidad. El tempo (BPM) y la clave armónica son métricas que se usan en la industria musical. El sistema usa estos datos para ofrecer información adicional.

Sistema de Notación Camelot. Método numérico que traduce las tonalidades tradicionales a un formato de fácil comprensión para la mezcla armónica. Esto facilita la selección de pistas compatibles armónicamente.

Single Source of Truth. Principio de arquitectura de datos que garantiza que cada pieza de información se almacene en un solo lugar.

Arquitectura Sin Estado (Stateless JWT). Modelo de comunicación donde el servidor no almacena información de la sesión del usuario. El uso de JSON Web Tokens (JWT) permite que cada petición sea independiente.

Autoalojamiento. Forma de despliegue donde el usuario mantiene la propiedad de la infraestructura y datos. A diferencia del modelo de software como servicio (SaaS), Musarum se ha diseñado para ser ejecutado en hardware privado, eliminando la dependencia de servidores de terceros y garantizando la privacidad.

Autoridad del Servidor (Server Authority). Principio de seguridad y diseño arquitectónico que establece al backend como el único validador legítimo.

2.2. Justificación del Proyecto

El desarrollo de Musarum responde a una necesidad técnica y funcional derivada de las limitaciones del modelo actual de consumo musical digital, que se estructura en torno a tres problemas concretos:

Dependencia y fragilidad de la propiedad digital. El modelo de Software como Servicio (SaaS) ha hecho que el usuario dependa por completo de licencias que no

controla. Si una discográfica retira un álbum, este desaparecerá de la lista del usuario sin un aviso previo. Además, se puede añadir la fragmentación del mercado musical, que obliga al pago de varias suscripciones para acceder a un catálogo completo. Musarum elimina esta dependencia.

Brecha funcional del software local: Los reproductores multimedia locales permiten conservar la propiedad de los archivos, pero ofrecen interfaces menos vistosas y funcionalidades más básicas. Las plataformas de streaming, por el contrario, proporcionan una experiencia agradable pero a costa de ceder el control de los datos. El usuario tiene que elegir entre propiedad y comodidad. Musarum plantea otra visión ofreciendo una interfaz moderna y funcionalidades sobre una audioteca propia.

Ausencia de analítica personal accesible: Las métricas de escucha y los resúmenes personalizados son funcionalidades habituales en las plataformas comerciales, pero dependen de terceros y puede ser funcionalidades premium. Musarum implementa su propia capa de datos mediante Spring Data JPA, permitiendo al usuario consultar sus hábitos de escucha de forma privada y gratuita.

Musarum se justifica como una herramienta que devuelve al usuario el control sobre su música, dotando a una audioteca local de las funcionalidades y la inteligencia de un servicio de streaming moderno.

3. OBJETIVOS

3.1. Objetivos Funcionales

Se definen las capacidades que el sistema debe ofrecer a los usuarios que se registre en Musarum.

Gestión de Roles y Autenticación. Implementar un sistema de registro e inicio de sesión, diferenciando el acceso mediante niveles jerárquicos de privilegios: Oyente, Contribuidor y Propietario.

Escaneo de Audioteca. Permitir al rol OWNER escanear una audioteca mediante el escaneo de directorios locales del servidor.

Gestión Remota de Archivos. Habilitar endpoints seguros para que usuarios con los roles autorizados puedan subir archivos de forma online, integrándolos en el catálogo.

Extracción y Enriquecimiento de Metadatos. Extraer etiquetas ID3 del fichero y usar APIs externas para el enriquecimiento.

Letras Sincronizadas. Integrar servicios externos de metadatos para buscar y mostrar letras sincronizadas durante la reproducción en directo con librerías externas.

Reproducción en Streaming. Permitir el consumo de pistas desde el navegador mediante envíos parciales por streaming, sin necesidad de descargar el fichero.

Métricas de Consumo. Registrar un historial de reproducciones para generar estadísticas de consumo en tiempo real.

Administración Avanzada. Dotar al rol OWNER de un panel de control con métricas del sistema y herramientas de moderación de usuarios.

Distribución Multiplataforma. Garantizar que el sistema sea accesible desde diversos entornos de ejecución.

- **Web:** interfaz adaptativa (Angular).
- **Desktop:** aplicación de escritorio mediante Electron.

3.2. Objetivos No Funcionales

Se definen los atributos de calidad, seguridad y eficiencia de la aplicación.

Despliegue y Portabilidad. Asegurar que la aplicación sea independiente de la infraestructura en la cual se desarrolle mediante la contenerización del servidor y de la base de datos utilizando Docker, facilitando su instalación en cualquier entorno.

Optimización y Rendimiento. Minimizar el consumo de memoria RAM en el servidor dejando los cálculos estadísticos al motor de la base de datos MariaDB, así como implementar en el sistema herramientas que aceleren las respuestas.

Seguridad y Control de Acceso. Proteger el acceso a los endpoints de Musarum mediante la implementación de JSON Web Tokens y garantizar la seguridad de las credenciales mediante algoritmos de cifrado de contraseñas.

Accesibilidad Remota. Garantizar el acceso seguro al servidor doméstico desde cualquier ubicación mediante una VPN, permitiendo el streaming entre los diferentes clientes y el servidor sin exponer puertos del router a Internet para evitar ataques.

Tolerancia a Fallos. Garantizar que un error en un proceso de la aplicación no interrumpa la usabilidad de la aplicación, mediante un control de excepciones propias.

Usabilidad y Diseño. Ofrecer una interfaz agradable y visualmente sencilla que permita al usuario navegar, buscar y reproducir contenido sin problemas, independientemente del dispositivo en el que se acceda.

4. HISTORIAS Y CASOS DE USO

4.1. Actores del Sistema

El sistema implementa un modelo de seguridad basado en control de acceso por roles, organizando a los usuarios en tres niveles jerárquicos.

En primer lugar, el **Oyente (LISTENER)**: perfil base. Posee permisos de lectura sobre el catálogo musical, reproducción multimedia y gestión de sus propios datos. En segundo lugar, el **Contribuidor (CONTRIBUTOR)**: perfil intermedio que hereda las capacidades de Oyente. Suma privilegios de escritura orientados a la aportación de contenido. Finalmente, el **Propietario (OWNER)**: rol de máxima autoridad que hereda todas las capacidades del **Contribuidor**. Asume el control sobre la infraestructura, la gestión de la comunidad e indexación del catálogo.

4.2. Historias de Usuario

4.2.1. Rol de Oyente o Listener

HU-01 (Registro): Como Oyente, quiero poder registrarme en el sistema introduciendo mis datos básicos, para crear una cuenta personal y acceder al catálogo musical.

HU-02 (Autenticación): Como Oyente, quiero iniciar sesión validando mi correo electrónico, para obtener un token de acceso que me identifique en el sistema.

HU-03 (Gestión de Perfil): Como Oyente, quiero modificar los datos de mi perfil o eliminar mi cuenta, para mantener el control sobre mi información personal.

HU-04 (Reproducción): Como Oyente, quiero reproducir pistas vía streaming desde cualquier cliente y ubicación, para disfrutar de la música sin descargar los ficheros.

HU-05 (Búsqueda Avanzada): Como Oyente, quiero realizar búsquedas y aplicar filtros dinámicos por título, artista o álbum, para localizar contenido específico.

HU-06 (Favoritos): Como Oyente, quiero marcar y desmarcar canciones como favoritas con un solo clic, para crear mi propia colección de acceso rápido.

HU-07 (Playlists): Como Oyente, quiero crear y gestionar listas personalizadas, para organizar el contenido multimedia según mis preferencias o estados de ánimo.

HU-08 (Analítica Personal): Como Oyente, quiero consultar mis métricas de escucha, para descubrir mis propios hábitos musicales.

4.2.2. Rol de Contribuidor o Contributor

HU-09 (Aportación de Contenido): Como Contribuidor, quiero subir mis propios archivos al servidor, para ampliar el catálogo global.

HU-10 (Control de Contenido Propio): Como Contribuidor, quiero tener permisos para eliminar canciones que he subido, para mantener el control sobre mis aportaciones.

HU-11 (Enriquecimiento): Como Contribuidor, quiero usar el enriquecimiento de metadatos de pistas, para corregir errores.

4.2.3. Rol de Propietario u Owner

HU-12 (Escaneo de Directorio Local): Como Propietario, quiero ejecutar un escáner automatizado sobre el directorio del servidor, para importar canciones de forma desatendida sin duplicar el peso físico en disco.

HU-13 (Gestión de Comunidad y Embargos): Como Propietario, quiero suspender o eliminar cuentas de usuarios, aplicando un proceso de embargo lógico (reasignación de sus pistas al propietario antes del borrado) para evitar la pérdida de canciones.

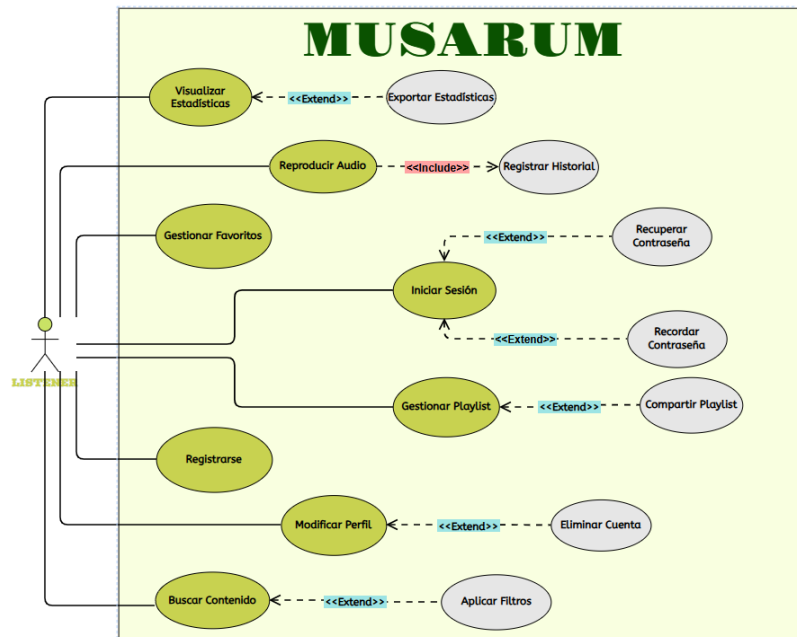
HU-14 (Mantenimiento de Catálogo): Como Propietario, quiero ejecutar un borrado del catálogo, para reiniciar la audioteca sin afectar a los registros de usuarios existentes.

HU-15 (Monitorización): Como Propietario, quiero visualizar un panel de control con métricas del hardware, para monitorizar la salud del servidor.

4.3. Casos de Uso

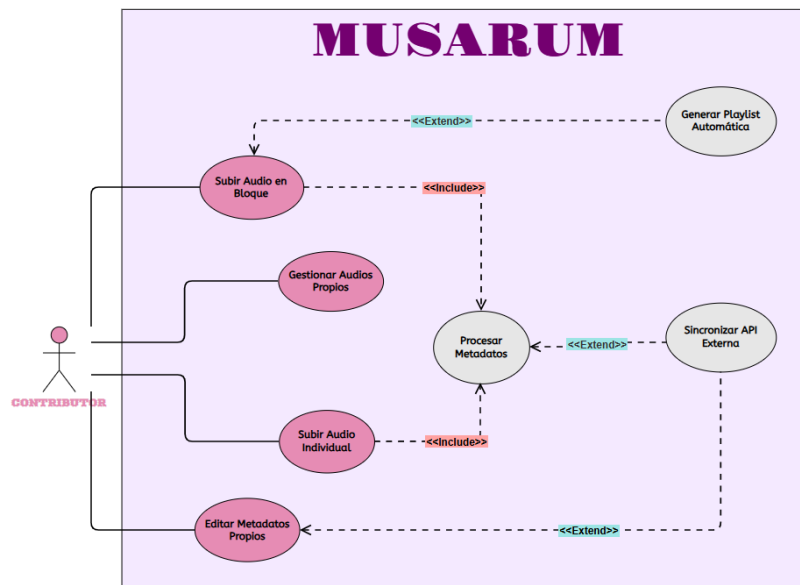
4.3.1. Rol de Oyente o Listener

El siguiente diagrama modela las interacciones disponibles para el rol Oyente/Listener, que constituye el perfil base del sistema. Sus casos de uso se centran en la lectura del catálogo, la reproducción de audio y la gestión de su propia cuenta. Cabe destacar que la reproducción de una pista desencadena de forma automática el registro en el historial de escucha, un proceso transparente para el usuario que alimenta el módulo de analítica personal.



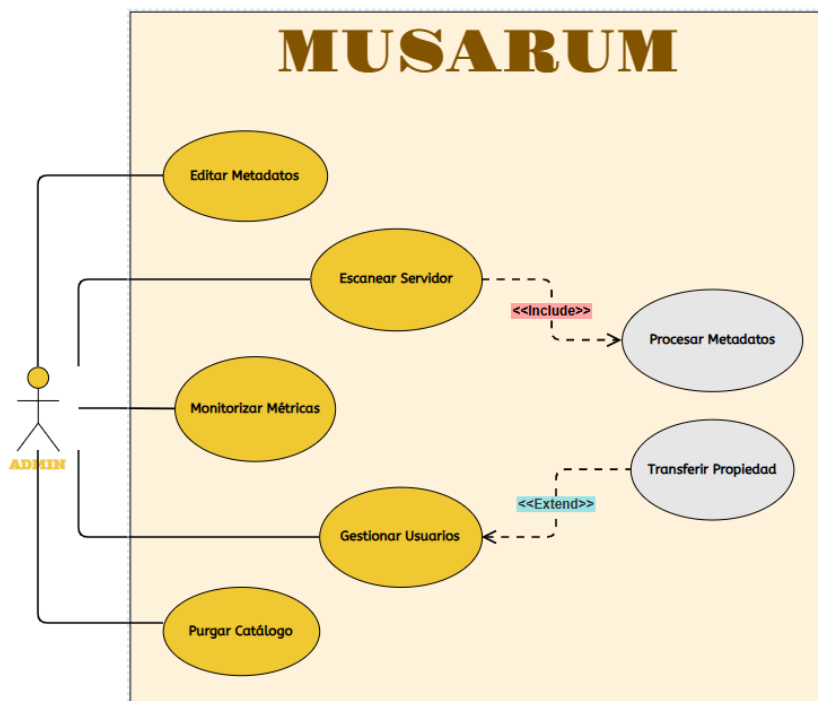
4.3.2. Rol de Contribuidor o Contributor

Este diagrama refleja las capacidades del rol Contribuidor/Contributor, que hereda todos los casos de uso del Listener y añade privilegios de escritura sobre el catálogo. La acción principal es la subida de archivos de audio, que activa de forma automática el procesamiento de metadatos en el servidor.



4.3.3. Rol de Propietario u Owner

Finalmente, el diagrama del Propietario u Owner ilustra las operaciones de administración y mantenimiento reservadas al rol de máxima autoridad. Destacan el escaneo del directorio local y herramientas de gestión de usuarios, incluyendo la suspensión de cuentas y el borrado con reasignación de contenido.



5. METODOLOGÍA

Musarum se ha realizado de forma individual, realizando las funcionalidades secuencialmente. El proceso consistía en diseñar primero el modelo de datos en la base de datos, programar después el backend para exponer esa información a través de la API y validar su funcionamiento con Swagger y Postman. El backend se construyó entero antes de iniciar el frontend en Angular, que se desarrolló siguiendo el mismo criterio: cada funcionalidad se completaba de principio a fin, corrigiendo errores en ambas capas a medida que surgían

El código se gestionó con Git y se subió de forma privada a GitHub. El repositorio contiene dos carpetas: `backend/` con el proyecto de Spring Boot, `web/` con la aplicación Angular, además de los ficheros de configuración de Docker en la raíz. Para trabajar de forma segura, el proyecto se ha ido desarrollando en diferentes ramas, creadas para realizar las diferentes funcionalidades. Al terminar la funcionalidad, se realizaba un *commit* con el mensaje correspondiente e inmediatamente un *merge* con la rama `main` y, finalmente, se realizaba un *push* al repositorio remoto. Esto me ha permitido evitar errores en la rama `main`, pudiendo probar en ramas externas funcionalidades sin miedo a destrozar funcionalidades ya activas y probadas. También se encuentra configurado el fichero `.gitignore` para que no se subieran carpetas o archivos sensibles.

GitHub ofrece tableros tipo Kanban para organizar tareas en columnas, pero no los utilicé a menudo. Al trabajar solo yo mismo me gestionaba las tareas y el propio flujo de dependencias entre funcionalidades de la misma aplicación actuó como planificación, con cada funcionalidad completada me surgía las siguientes que eran necesarias implementar o, simplemente, surgían a medida que avanzaba.

Las herramientas que utilicé fueron Spring Tools Suite para programar el backend, Visual Studio Code para el frontend, DBeaver para ver y administrar la base de datos MariaDB, Postman para probar los endpoints antes de conectarlos con la interfaz, Docker Desktop para gestionar los contenedores, y Swagger UI para tener la documentación de la API sin tener que escribirla yo mismo a mano.

El desarrollo del proyecto comenzó a finales de enero de 2026. Al principio, Musarum no pretendía ser como se describe en esta memoria, sino como una réplica funcional de Spotify. Esa idea se descartó al no tener mucho sentido: no existía una

lógica de negocio diferenciadora, no habría usuarios reales y no tenía capacidad de competir con plataformas consolidadas como Deezer, Apple Music o Spotify. Muy posiblemente se convertiría en un proyecto de TFG más, hoy quiero hacer de Musarum mi repositorio musical privado y hecho por mí mientras aprendo.

Entre el 1 y el 12 de febrero de 2026, aún sin una identidad clara para el proyecto, se diseñaron las entidades principales de la plataforma (*songs*, *albums*, *artists*, *genres*). Para decidir qué información debía guardar cada entidad, se analizaron las plataformas mencionadas, identificando los campos más importantes: tonalidad, BPM, duración, título, artista, género y álbum. Cabe destacar un patrón común en todas ellas que Musarum también copia: todas las canciones, incluso un single, pertenece a un álbum (aunque sea de una sola pista), lo que simplifica la lógica al garantizar que la relación canción-álbum siempre existe.

El concepto definitivo del proyecto surgió al reflexionar sobre la experiencia de uso de un reproductor local en mi dispositivo móvil Huawei: la música estaba ahí, pero la interfaz no estaba ni cerca de las de streaming actuales. La idea de construir un reproductor para una audioteca propia, con las funcionalidades de una plataforma moderna bajo el control total del usuario se formó como la propuesta que finalmente define a Musarum a día de hoy.

El desarrollo del backend ocupó todo el mes de marzo y gran parte de abril, implementando los endpoints, servicios y consultas sobre la API REST. Ya con el backend funcionando, se inició el desarrollo del frontend en Angular, estructurado por componentes. Al igual que ocurrió con la identidad conceptual del proyecto, la identidad visual también atravesó una fase inicial débil: el primer Wireframe era idéntico a la interfaz de Spotify. La decisión de alejarse de los reproductores comerciales me llevó a buscar una identidad visual propia (gracias a los azulejos del patio de mi casa), que finalmente se encontró en la arquitectura andalusí y mudéjar, adoptando una paleta de tonos oscuros con ornamentación geométrica en oro envejecido, fondo ink, textos en crema y resaltado con terra.

El resto del desarrollo, hasta una semana antes de la entrega, se dedicó a la creación de las Pages (Angular) principales (login, registro, artistas, canciones, géneros), la implementación de la seguridad en el cliente (guards e interceptors) y la configuración del enrutamiento.

Con ambas capas ya funcionales, pude proceder con la comunicación entre cliente y servidor a través de la VPN, instalando Tailscale y verificando el acceso mediante las direcciones IP estáticas de la red privada. Para garantizar disponibilidad durante la defensa, se han configuraron tres instancias del backend: la original en el servidor doméstico de Lebrija, una réplica en localhost y otra contenerizada en Docker. De igual forma, el frontend se empaquetó como aplicación de escritorio mediante Electron, que integra los archivos compilados de Angular en un motor Chromium y permite servir la aplicación desde el escritorio como si fuera una aplicación nativa.

La última semana se dedicó a la redacción de esta memoria, y el fin de semana previo a la defensa, a la preparación de la presentación.

6. ARQUITECTURA DEL SISTEMA

6.1. Visión General

Musarum se ha diseñado con arquitectura cliente – servidor de tipo RESTful sin estado. El servidor expone una API REST consumida por los clientes: aplicación web y de escritorio.

La comunicación entre cliente y servidor se realiza mediante peticiones HTTP en formato JSON. El servidor no mantiene sesiones en memoria, por lo que cada petición es independiente y debe transportar la información para identificar al usuario. Esta identificación se crea mediante un token JWT que el cliente añade en la cabecera *Authorization* de cada solicitud. Antes de alcanzar la capa de controladores, el token se valida criptográficamente por un filtro de seguridad, que verifica su firma y el usuario.

Esta arquitectura ofrece una independencia directa entre cliente y servidor, que se comunican solo con plantillas de datos predefinidos (DTO). Además, el backend stateless facilita la escalabilidad al no requerir sincronización entre instancias. La infraestructura y flujo de comunicación se muestran en el siguiente diagrama:

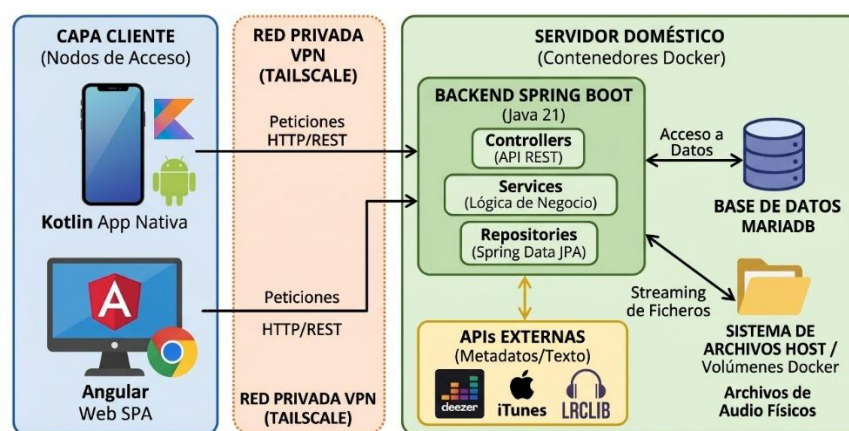


Ilustración 1. Diagrama de Componentes Global y Flujo de Comunicación

La arquitectura se distribuye en cuatro bloques. La capa cliente está compuesta por las aplicaciones web y escritorio (también móvil con responsive) que actúan como interfaces. Las peticiones que envían viajan a través de una VPN, que cifra el tráfico y permite el acceso al servidor doméstico desde el exterior sin exponer puertos. El servidor, alojado en Docker, contiene la lógica de negocio. Finalmente, la capa de persistencia combina una base de datos relacional, encargada de almacenar los

metadatos y relaciones, con un sistema de volúmenes donde están físicamente los ficheros de audio que se transmiten vía streaming.

6.2. Diseño Del Servidor

6.2.1. Arquitectura Por Capas

El backend se ha estructurado por capas, donde cada nivel tiene una responsabilidad y se comunica solo con los niveles colindantes. Las principales capas son: *controladores*, *servicios*, *repositorios* y *entidades*, complementadas por una capa de *transferencia* (DTO) que actúa como contrato entre el servidor – cliente.

La capa de controladores es la frontera del servidor. En Musarum se han implementado controladores organizados por dominio funcional (*autenticación*, *música*, *audioteca*, *búsqueda*, *usuarios* y *administración*). Su responsabilidad es recibir las peticiones HTTP, validar parámetros de entrada y delegar la petición al servicio. Todos los endpoints exigen un token JWT en la cabecera, salvo las rutas públicas de registro y login. Para la paginación y ordenación se usa Pageable de Spring Data, que permite respuestas fragmentadas sin sobrecargar al cliente.

La capa de servicios contiene la lógica del sistema. Los controladores no interactúan directamente con las clases lógicas, sino a través de interfaces con implementaciones que residen en clases separadas. Cada operación que afecta a la base de datos se ejecuta dentro de una transacción gestionada con `@Transactional`: si una operación falla a mitad del proceso, Hibernate revierte todos los cambios. Las operaciones de lectura tienen `@Transactional(readOnly = true)`, lo que desactiva el Dirty Checking, reduciendo el consumo de memoria y acelerando la respuesta. La capa de servicios también gestiona la caché mediante `@Cacheable` y `@CacheEvict`, que permiten almacenar en memoria resultados frecuentes y evitar consultas repetidas.

La capa de repositorios se ha implementado con Spring Data JPA, delegando la comunicación con MariaDB a través de interfaces que extienden `JpaRepository`. Para evitar el problema *N+1* en relaciones complejas, los repositorios utilizan la anotación `@EntityGraph`, que fuerza a Hibernate a recuperar una entidad junto con sus relaciones en una única consulta con JOIN.

Finalmente, la capa de entidades representa el modelo de dominio. Cada entidad se existe directamente como una tabla de la base de datos mediante JPA, y todas extienden una clase que gestiona la auditoría.

El recorrido completo de una petición HTTP sigue un patrón básicamente lineal. Cuando un cliente envía una solicitud, esta pasa primero por el filtro *RequestTimingFilter* para auditoría. A continuación, el *JwtAuthenticationFilter* extrae el token de la cabecera, valida su firma e inyecta el usuario en Spring. Ya autenticada, la petición pasa al controlador, que valida los parámetros de entrada, comprueba los permisos del endpoint y delega la operación al servicio. El servicio ejecuta la lógica bajo transaccionalidad, interactuando con los repositorios. Las entidades devueltas por la base de datos se transforman en DTO mediante clases Mappers, garantizando que la entidad de dominio no se exponga. Finalmente, el controlador envuelve el DTO en un *ResponseEntity* con el código HTTP y devuelve la respuesta como JSON.

En caso de que se produzca un error en el recorrido, un interceptor global captura y devuelve un mensaje de error gestionado con enums.

6.2.2. Variables De Entorno Y Configuración

La configuración del servidor se encuentra en el fichero *application.properties*, donde se centralizan los parámetros que afectan a la aplicación. Para evitar la exposición de credenciales, ningún valor se escribe en el código. En su lugar, el fichero actúa como una plantilla que referencia variables de entorno. Estas se ponen en el IDE en desarrollo y, en producción, se inyectan en el contenedor Docker en un fichero *.env*.

VARIABLE	USO EN EL SISTEMA
DB_URL	Conexión al motor de base de datos.
DB_USERNAME	Credencial de usuario de la base de datos.
DB_PASSWORD	Contraseña de acceso a la base de datos.
MAIL_USERNAME	Dirección de correo para validar cuenta.
MAIL_PASSWORD	Contraseña de aplicación para el correo.

JWT_SECRET	Clave para firmar y validar tokens JWT.
BASE_URL	URL dinámica del servidor.

La configuración también define parámetros de comportamiento del servidor: La política de seguridad desactiva la protección CSRF, innecesaria al tratarse de una API stateless, y restringe el acceso CORS a sólo los frontend de Musarum.

6.2.3. Transferencia De Datos

Para aislar el modelo de dominio interno del contrato público expuesto a los clientes, toda la comunicación entre el servidor y el frontend se realiza a través de objetos de transferencia de datos (DTO). Esta capa intermedia evita exponer entidades JPA con información sensible.

Los DTO de Musarum se han organizado en dos categorías semánticas: las clases de tipo Request representan los datos que el cliente envía al servidor y las Response, que recogen las respuestas que el servidor devuelve al cliente.

La transformación entre las entidades JPA y los DTO y viceversa se gestiona mediante Mappers, encargados de convertir las entidades en su correspondiente DTO de respuesta y viceversa.

6.3. Diseño Del Cliente Web

El cliente web se ha desarrollado con Angular 21 utilizando la arquitectura de componentes. La aplicación se estructura en carpetas funcionales, cada una con una responsabilidad: *components/* para elementos reutilizables, *pages/* para las pantallas completas de la aplicación, *guards/* para la protección de rutas, *interceptors/* para la manipulación de peticiones HTTP, *services/* para la comunicación con la API, *models/* para las interfaces y *layout/* para la estructura visual común.

A diferencia del backend, que se organiza por capas técnicas, el frontend agrupa el código por funcionalidad. Cada pantalla reside en su propia carpeta dentro de *pages/*, lo que facilita localizar y modificar cualquier parte de la interfaz.

El enrutamiento se define en *app.routes.ts* mediante *lazy loading*, de modo que cada pantalla solo se carga cuando el usuario accede a ella por primera vez. Las rutas se dividen en dos grupos: rutas públicas accesibles sin autenticación y rutas privadas que requieren sesión activa, todas ellas bajo el componente *MainLayout*.

La seguridad del enrutamiento se gestiona mediante dos guards. El primero, *authGuard*, intercepta cualquier intento de acceso a rutas protegidas y verifica si existe un token JWT válido en el almacenamiento local del navegador. Si el token no existe o ha expirado, el guard redirige a la pantalla de login. El segundo, *roleGuard*, aplica un nivel de protección basado en roles.

El envío automático del token JWT en cada petición se resuelve mediante un interceptor HTTP, que captura todas las solicitudes salientes hacia la API y, si existe un token almacenado, lo inyecta en la cabecera.

La configuración se centraliza en *environment.ts*, donde residen las variables *apiUrl* que definen la dirección base del servidor. Durante el desarrollo apunta a localhost pero puede cambiarse a la IP con Tailscale del servidor doméstico.

6.4. Diseño De Persistencia

6.4.1. Diagrama Entidad – Relación

La persistencia se gestiona con una base de datos relacional MariaDB, modelada mediante entidades JPA del backend. Existen cuatro grandes dominios.

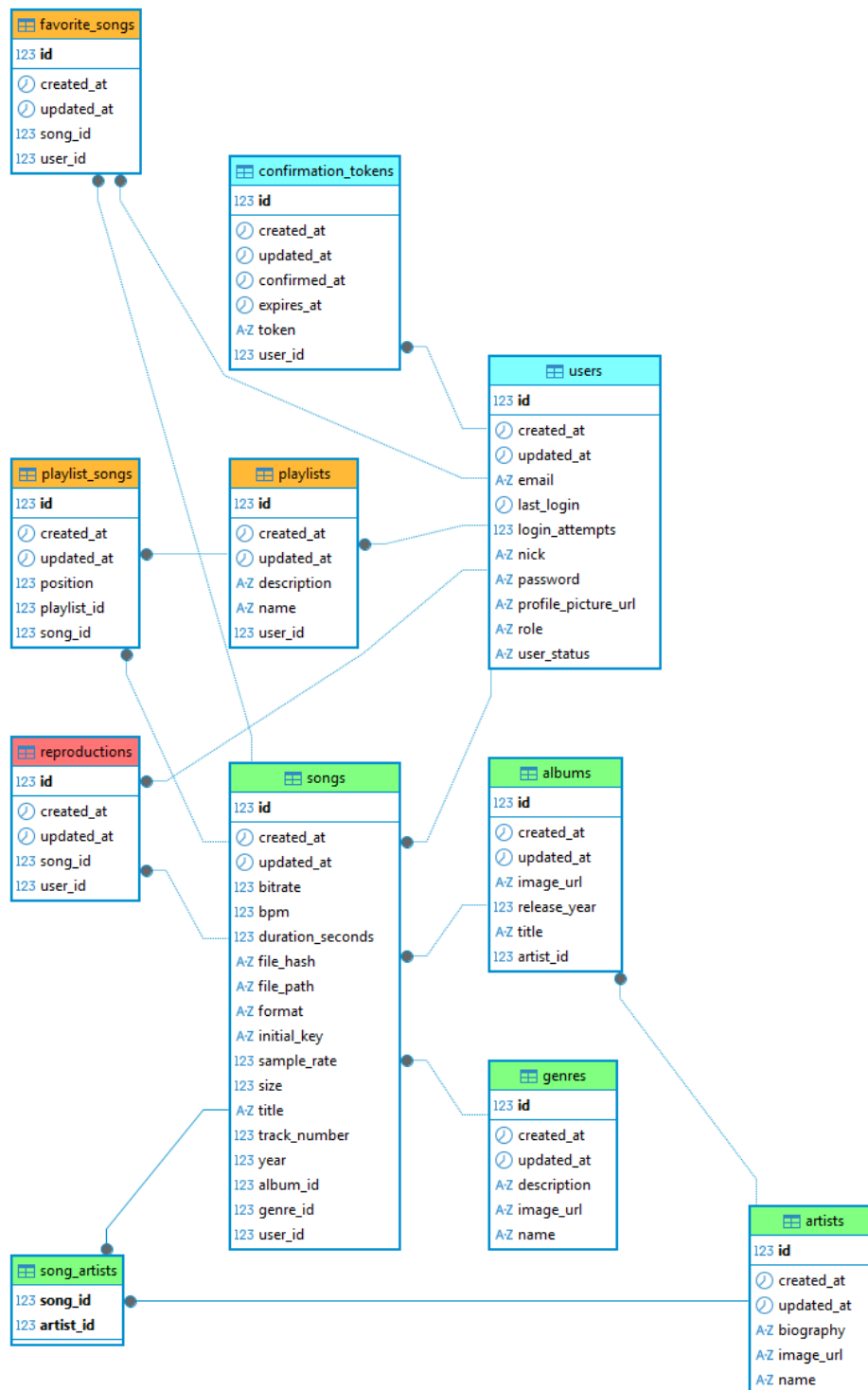
El dominio de **identidad y seguridad**, compuesto por la tabla *users*, centraliza la información de las cuentas, incluyendo el rol y estado. Esta tabla se complementa con *confirmation_tokens*, que almacena los tokens generados.

El dominio de **catálogo musical** constituye el núcleo del sistema. La tabla central es *songs*, que almacena la información técnica. Esta entidad se relaciona con tres tablas descriptivas: *albums*, *artists* y *genres*. Dado que una canción puede contar con múltiples colaboradores, se incorpora la tabla puente *song_artists*.

El dominio de **audioteca de usuario** modela las colecciones. La tabla *playlists* almacena información de las listas de reproducción, mientras que *playlist_songs* actúa como tabla puente entre playlists y canciones. Esta última incluye un atributo position

que permite conservar el orden de la pista. La tabla *favorite_songs* registra las pistas marcadas como favoritas.

Finalmente, el dominio de **métricas de usuario**: *reproductions*, encargada de registrar los eventos de escucha. Cada fila almacena el usuario, la canción y la fecha. Proporciona los cálculos estadísticos.



6.4.2. Integridad Referencial

En primer lugar, se aplica el borrado lógico en el dominio de identidad. El sistema modifica el estado de la cuenta. Por ejemplo, cuando un usuario supera los intentos fallidos de login o el propietario quiere bloquearlo, su estado pasa a BANNED, revocando el acceso pero preservando su información. Solo el administrador puede devolverle el status de Active.

En segundo lugar, se implementa el borrado físico en cascada controlado por transacciones. Cuando un usuario elimina una de sus listas, el servicio elimina previamente todas las entradas dependientes en `playlist_songs`, evitando errores de clave foránea.

En tercer lugar, se aplica la reasignación de datos huérfanos cuando el propietario elimina físicamente a un colaborador. Antes de eliminar, las pistas que ese usuario subiera se asignan al propietario evitando la pérdida de contenido.

6.4.3. Almacenamiento De Ficheros Físicos

Aunque la base de datos centraliza los metadatos, los ficheros de audio e imágenes no se almacenan ahí ya que al persistir archivos físicos disminuye el rendimiento y, por este motivo, Musarum sólo almacena rutas relativas a los recursos, mientras que el contenido físico reside en el host, expuesto al contenedor Docker a través de un volumen.

El directorio raíz de almacenamiento se define mediante la propiedad `musarum.storage.path` y, por defecto, se ubica en la carpeta `.musarum` del host. Dentro de este directorio raíz se han establecido subcarpetas:

CARPETA	USO EN EL SISTEMA
Upload/	Pistas subidas por usuarios, organizadas por nick. (No entran aquí las escaneadas)
Avatars/	Imágenes de perfil de los usuarios.
Lyrics	Letras de pistas.

6.5. Patrones De Diseño

Se han aplicado varios patrones de diseño, todos ellos integrados en el ecosistema de Spring y complementados por anotaciones Lombok.

El primero de ellos es la **inyección de dependencias**. En lugar de que cada clase instancie sus dependencias manualmente, Spring se encarga de proporcionarlas en tiempo de ejecución. La inyección se realiza a través del constructor, generado por Lombok sobre los campos final. Este enfoque permite delegar en Spring la creación, vida útil y destrucción de las dependencias.

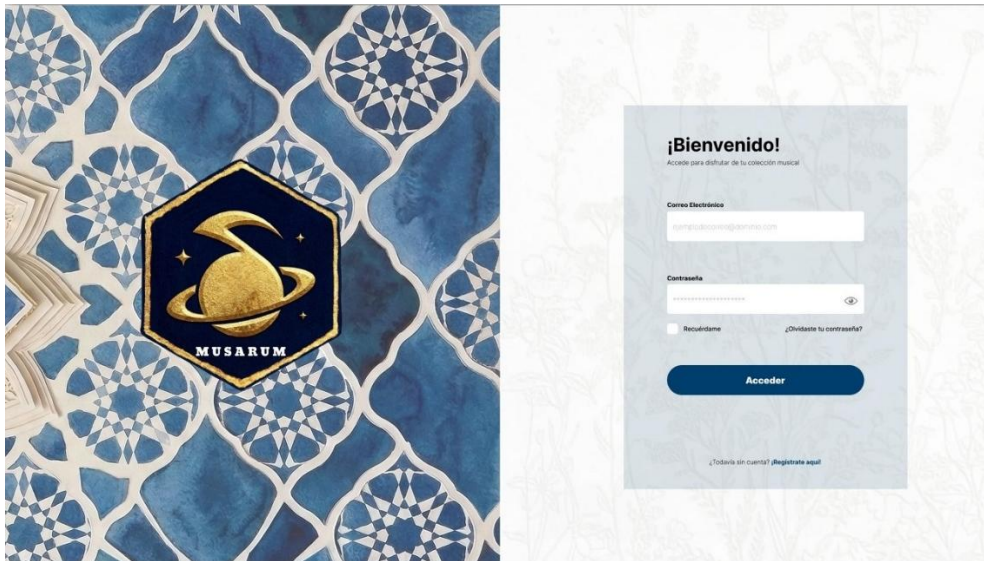
Para la construcción de objetos se ha optado por el **patrón Builder** de Lombok. Este patrón sustituye los constructores extensos por una cadena de atributos.

El patrón **Data Transfer Object**, ya mencionado en apartados anteriores, se aplica para desacoplar el modelo de dominio de lo expuesto a los clientes.

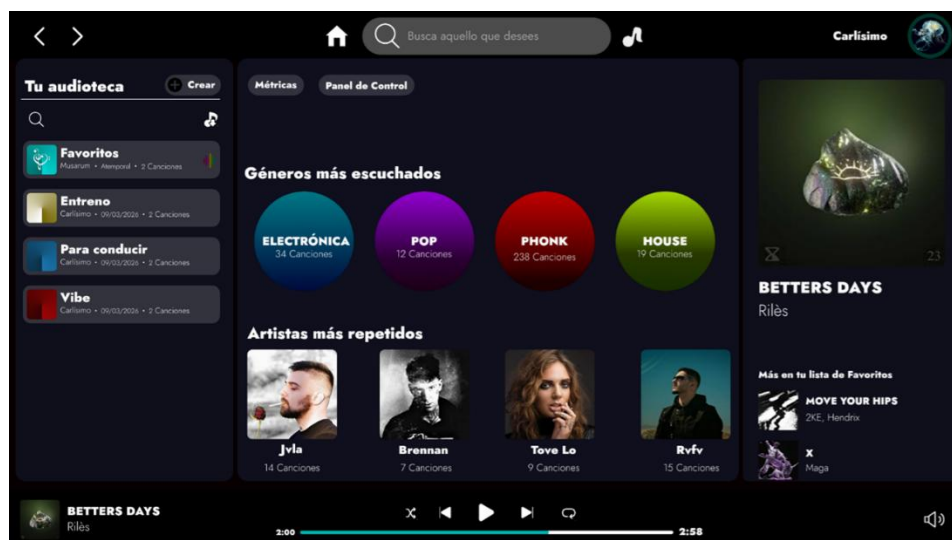
Por último, el acceso a datos se gestiona mediante el **patrón Repository** a través de Spring Data JPA. Cada entidad tiene una interfaz que extiende JpaRepository. El framework genera las consultas a partir de los nombres de los métodos.

7. DISEÑO DE INTERFAZ

Durante el desarrollo, el diseño inicial evolucionó hacia una identidad propia más diferenciada. La propuesta original presentaba similitudes estéticas con plataformas comerciales como Spotify. La gran similitud evitó continuar en esa línea y cambiar hacia una estética inspirada en la arquitectura andalusí. A continuación se muestra el diseño de la pantalla de Acceso a la plataforma, diseño finalmente descartado por falta de personalidad.



En la siguiente figura, se observa la siguiente pantalla que el usuario vería al acceder con sus credenciales de usuario.



El diseño evolucionó hacia una identidad diferenciada, alejándose de la estética plana y genérica habitual en las aplicaciones de streaming. La interfaz actual combina un modo oscuro con elementos visuales inspirados en la ornamentación geométrica

andalusí y mudéjar. La paleta cromática se construye sobre tonos negros, grises y dorados envejecidos, más propio de una audioteca personal y cuidada. A continuación se muestran las pantallas principales que materializan esta experiencia.

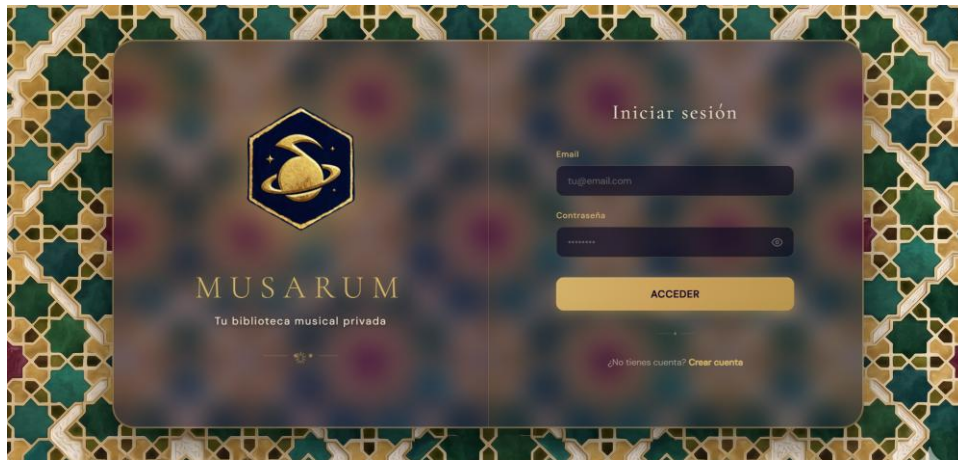


Ilustración 2. Página de Acceso de Musarum

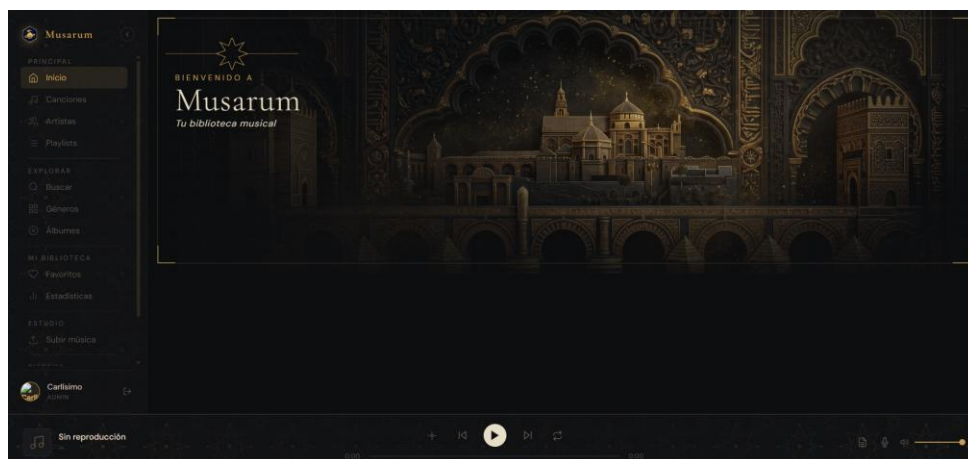


Ilustración 3. Página de Inicio

Se añade el logo de la plataforma. Hace referencia a una nota musical con anillos periféricos, haciendo alusión a un planeta, el mundo musical que cuida y organiza toda tu audioteca en un mismo lugar.



8. IMPLEMENTACIÓN TÉCNICA

8.1. Stack Tecnológico

8.1.1. Backend

HERRAMIENTA	TECNOLOGÍA
Lenguaje	Java 21 LTS
Framework	Spring Boot 4.0.6
Persistencia	Spring Data JPA + Hibernate
Seguridad	Spring Security + JWT 0.11.5
Documentación API	SpringDoc OpenAPI 2.6.0 (Swagger UI)
Caché	Caffeine
Validación	Jakarta Validation

8.1.2. Frontend

HERRAMIENTA	TECNOLOGÍA
Lenguaje	TypeScript
Framework	Angular 21
Maquetado	HTML5, SCSS
Cliente HTTP	HTTPClient de Angular

8.1.3. Persistencia

HERRAMIENTA	TECNOLOGÍA
Motor de Base de Datos	MariaDB
Administración Visual	DBeaver
Almacenamiento de Ficheros	En el host (Volúmenes Docker)

8.1.4. Librerías Auxiliares

HERRAMIENTA	TECNOLOGÍA
Lombok	Reducción de código repetitivo
Mp3AgiC	Lectura de metadatos ID3v2 en MP3
JJWT	Generación y validación de Tokens
Caffeine	Caché en memoria
Spring DevTools	Reinicio automático en desarrollo

8.1.5. APIs Externas

HERRAMIENTA	TECNOLOGÍA
Deezer	Portadas, fotografía de artista, géneros.
MusicBrainz	Número de pista
iTunes Search API	Año de publicación.
TheAudioDB	BPM
LRCLib	Letras planas y sincronizadas.

8.1.6. Herramientas de Desarrollo y Despliegue

HERRAMIENTA	TECNOLOGÍA
IDE Backend	Spring Tools Suite
IDE Frontend	Visual Studio Code
Control de Versiones	Git, GitHub (Repositorio)
Pruebas de API	Postman
Contenerización	Docker, DockerHub
Red Privada Virtual	TailScale

8.2. Seguridad

La protección del sistema se fundamenta en: **autenticación de usuarios, control de permisos** sobre los recursos y la **validación de datos entrantes**.

La autenticación se resuelve mediante tokens JWT y una clave secreta inyectada desde una variable de entorno. Cuando un usuario inicia sesión, el servidor genera un token que incorpora en su cuerpo varios claims personalizados (identificador interno, rol jerárquico y nombre de usuario), evitando así consultas innecesarias a la base de datos en cada petición. El cliente almacena este token y lo adjunta en la cabecera Authorization de cada solicitud.

El control de acceso se gestiona a nivel de método mediante la anotación @PreAuthorize de Spring Security, encapsulada en anotaciones propias. Las anotaciones @IsAdmin y @IsAdminOrContributor se aplican sobre los métodos que requieren privilegios.

Por último, la validación de los datos entrantes. Los DTO de tipo Request declaran restricciones mediante anotaciones de Jakarta Validation (@NotBlank, @Email, @NotNull, @Pattern), forzando que las contraseñas tengan características específicas, o que el nick respete un formato alfanumérico.

8.3. Funcionalidades Principales

8.3.1. Gestión de Usuarios y Autenticación

El módulo cubre el ciclo de vida completo de una cuenta: registro, confirmación, inicio de sesión y gestión del perfil.

El registro verifica que el correo y el nick no estén en uso, encripta la contraseña con BCrypt y crea la cuenta con estado PENDING_CONFIRMATION. El primer usuario registrado recibe automáticamente el rol OWNER; el resto, LISTENER.

El inicio de sesión valida que la cuenta esté activa, comprueba la contraseña con BCrypt e incrementa un contador de intentos fallidos en cada error. Tras cuatro fallos consecutivos la cuenta pasa a BANNED. tras un login correcto, el contador se reinicia y se devuelve el token JWT junto con los datos del usuario.

La gestión del perfil permite consultar y modificar parcialmente los datos personales.

Las operaciones administrativas se exponen con `@IsAdmin`, permitiendo listar usuarios, cambiar su estado o eliminarlos físicamente. El sistema impide que el propietario se elimine a sí mismo.

8.3.2. Carga y Gestión de Audioteca Musical

La incorporación de música al sistema se realiza por dos vías: la subida manual y el escaneo de un directorio local.

La subida manual acepta hasta 50 ficheros por lote. Para cada uno el sistema valida que no exista. El escaneo del directorio local está reservado al propietario y permite escanear una carpeta completa del host sin moverlas de su ubicación. El servicio recorre la ruta indicada y procesa todos los ficheros.

Durante el procesamiento, el progreso se expone en tiempo mediante un contador `AtomicInteger` que garantiza la seguridad en entornos concurrentes y permite al cliente consultar el avance. Tanto la subida manual como el escaneo devuelven un objeto `BatchResponse` con el número de pista exitosas y fallidas. El listado de canciones disponibles se obtiene con paginación nativa y ordenación por título.

8.3.3. Streaming de Audio

La transmisión de audio se ha implementado siguiendo el estándar `HTTP Range Requests`, soportado de forma nativa por la etiqueta `<audio>` de `HTML5`. Permite al cliente solicitar fragmentos concretos de un fichero en lugar de descargarlo completo, habilitando funcionalidades como el salto temporal (`seek`) instantáneo.

Cuando el cliente emite una petición sin cabecera `Range`, el servidor responde con un código 200 y transmite el fichero completo. Cuando la petición incluye una cabecera del tipo `Range` el servidor responde con el código 206 `Partial Content`, calcula los desplazamientos solicitados y devuelve los bytes del rango especificado.

8.3.4. Enriquecimiento de Datos

Tras la incorporación de una canción al sistema, el servicio orquesta un proceso de enriquecimiento automático que completa los metadatos faltantes consultando APIs

externas. El orden de consulta y el alcance dependen del ScanMode seleccionado. El comportamiento del escaneo se controla mediante un enum ScanMode que define tres niveles: FAST (metadatos locales), STANDARD (Deezer) y FULL (todas las APIs disponibles).

La primera y más relevante es la API de Deezer, que aporta la portada del álbum, la fotografía del artista y el género musical asociado. Las demás APIs solo rellenan aquellos metadatos que Deezer deja en blanco al no obtener resultados, como por ejemplo: año de publicación, el número de pista, etc. Todas las APIs consultadas son gratuitas para uso no comercial.

Finalmente, LRCLib se consulta para obtener las letras de las canciones. Esta funcionalidad permite la visualización de letra sincronizada de forma sencilla.

8.3.5. Listas de Reproducción y Favoritos

El módulo permite a cada usuario organizar su audioteca personal mediante dos mecanismos complementarios: listas de reproducción ordenadas y un sistema rápido de marcado de favoritos.

La gestión de playlists se expone a través de un conjunto de endpoints CRUD bajo /playlists. El usuario puede crear una nueva lista, consultar las suyas, acceder a los detalles o eliminarla. La adición de canciones persiste cada entrada en la tabla puente playlist_songs junto con su posición secuencial. Esta posición permite al usuario reordenar manualmente las canciones. Todas las operaciones verifican previamente la propiedad de la lista de reproducción.

El sistema de favoritos es más directo para destacar canciones. El endpoint actúa como toggle: si la canción ya estaba marcada, la elimina de la tabla favorite_songs; en caso contrario, la añade. La unicidad de la combinación usuario–canción se garantiza mediante una restricción UNIQUE a nivel de base de datos, evitando duplicados.

8.3.6. Modos de Reproducción Avanzados

Más allá de los modos clásicos de aleatorio y bucle, Musarum incorpora modos de reproducción que utilizan los metadatos enriquecidos (Tonalidad y BPM) para construir transiciones coherentes entre canciones. Estos modos pretenden simular el

comportamiento que aplicaría un DJ al encadenar pistas, ofreciendo una experiencia de escucha más fluida que la reproducción aleatoria tradicional.

Los modos básicos son tres: *aleatorio* (selección uniforme entre las canciones disponibles), *repetición de canción* (la pista actual vuelve a reproducirse al terminar) y *repetición de lista* (al llegar a la última canción, la reproducción vuelve a la primera).

Se han implementado los modos de *tonalidad* y *tempo*, que filtran las canciones siguientes en función de la pista que se está reproduciendo. En el plano armónico, los modos aumentar, mantener y disminuir seleccionan canciones cuya tonalidad encaja con la actual según las reglas de la rueda de Camelot, una notación estándar en el mundo del DJ. La utilidad CamelotUtil se encarga de traducir las tonalidades estándar (A Minor, C Major, etc.) al formato Camelot (8A, 8B, etc.) durante el mapeo de la pista.

En el plano rítmico, los modos aumentar, mantener y disminuir BPM seleccionan como siguiente pista aquella cuyo tempo se encuentre dentro de un margen tolerable respecto al actual.

8.3.7. Reconocimiento de Voz

Como predecesor de un sistema de control por voz más completo, Musarum incorpora un módulo muy básico de reconocimiento sobre la API WebSpeech del navegador, que aprovecha el motor de reconocimiento nativo del cliente.

La implementación actual reconoce tres comandos esenciales (parar, siguiente y anterior) que el usuario puede pronunciar para controlar la reproducción sin manipular la interfaz.

Esta funcionalidad primaria sienta las bases sobre las que se podrá construir un modo coche completo, con vocabulario ampliado para invocar canciones, álbumes o playlists concretas mediante lenguaje natural.

8.3.8. Búsqueda Global

El sistema ofrece un buscador unificado capaz de explorar simultáneamente las tres entidades principales del catálogo (canciones, artistas y álbumes). Internamente, la búsqueda se ejecuta sobre los tres repositorios mediante consultas insensible a

mayúsculas. Cada repositorio devuelve un conjunto limitado de resultados. La respuesta agrupa los tres conjuntos dentro de un único objeto SearchResponse.

8.3.9. Estadística y Analítica

El sistema registra cada reproducción en la tabla reproductions y aprovecha esta información para ofrecer al usuario un conjunto de métricas personales sobre sus hábitos de escucha.

El endpoint devuelve un resumen general con el total de reproducciones, los segundos acumulados de escucha, el número de artistas y canciones únicas, y los rankings personales de los cinco artistas, géneros y canciones más reproducidos. También se ofrece un calendario que agrupa las reproducciones por día durante el último año.

Por último, se identifica la hora más activa del día y el día de la semana en que el usuario más escucha música, utilizando las funciones HOUR() y DAYOFWEEK() directamente sobre la columna created_at.

8.3.10. Administración del Sistema

El módulo de administración agrupa las operaciones reservadas al rol PROPIETARIO para gestionar el estado global de la plataforma. Todas sus rutas se están protegidas mediante la anotación @isAdmin.

La gestión masiva del catálogo se centra en dos endpoints. POST /admin/library/scan inicia el escaneo de un directorio local del host, mientras que DELETE /admin/library/wipe ejecuta un borrado completo del contenido musical del sistema. Esta segunda operación solo conserva los usuarios. La operación utiliza deleteAllInBatch para evitar la carga de las entidades.

El endpoint GET /admin/library/metrics ofrece una visión agregada del sistema, devolviendo el total de canciones, álbumes y artistas, el tamaño de la audioteca.

Por su parte, GET /admin/library/health calcula un índice de salud de la audioteca entre 0 y 100, evaluando el porcentaje de canciones con portada de álbum, con fotografía de artista, con género válido.

8.4. Optimizaciones de Rendimiento

Durante el desarrollo se han incorporado técnicas para garantizar tiempos de respuesta bajos y un consumo de recursos contenido.

En primer lugar, el sistema integra **Caffeine** como **gestor de caché** en memoria, configurada con una expiración de quince minutos y un máximo de quinientas entradas. Se han definido dos cachés principales: `topSongs`, que almacena las canciones más reproducidas para evitar recalcularlas en cada petición, y `jwtUsers`, que mantiene los usuarios autenticados accesibles desde el filtro JWT sin necesidad de consultar la base de datos en cada solicitud. Las anotaciones `@Cacheable` y `@CacheEvict` gestionan el ciclo de vida de las entradas, invalidando los datos cuando ocurren eventos de modificación o creación.

En el plano de la base de datos, las tablas principales declaran **índices estratégicos** sobre las columnas más consultadas. La tabla `songs` los aplica sobre `title`, `genre_id`, `album_id` y `user_id`. La tabla `users` cuenta con índices sobre `email` y `nick`, consultadas en el proceso de autenticación.

Las relaciones entre entidades JPA se cargan por defecto en modo **lazy**, evitando que cada consulta arrastre claves foráneas. Para las consultas en las que sí se necesita información asociada, se utiliza la anotación `@EntityGraph` indicando qué relaciones deben aparecer en la consulta. Esta técnica resuelve el **problema N+1**, forzando a Hibernate a generar un único *join* en lugar de ejecutar una consulta adicional por cada entidad relacionada.

Todos los endpoints que devuelven listados usan **Pageable** que Spring Data traduce en cláusulas *limit* y *offset* en la consulta SQL.

Por último, para las consultas analíticas con pocos campos, se ha implementado la interfaz `TopItemProjection`, que expone únicamente los métodos `getName()` y `getCount()`. Spring Data instancia automáticamente **proyecciones** que cumplen esta interfaz a partir de los resultados de las consultas, evitando la carga completa de las entidades y reduciendo el consumo de memoria.

9. DESPLIEGUE Y CONFIGURACIÓN

El despliegue de Musarum combina contenerización con Docker y conectividad remota, permitiendo ejecutarlo en servidores domésticos accesibles desde cualquier ubicación sin exponer puertos a internet.

La configuración del entorno parte del archivo `.env` que almacena las variables sensibles y se excluye del control de versiones. Su complemento, `.env.example`, es una plantilla que facilita la configuración a quien replique el proyecto.

La contenerización se gestiona mediante `docker-compose.yml`, que orquesta dos servicios. El primero, `musarum-mariadb`, utiliza la imagen oficial de MariaDB. El segundo, `musarum-backend`, se construye a partir de un `Dockerfile`, que copia el archivo compilado al contenedor. La dependencia explícita entre servicios garantiza que la base de datos esté operativa antes de que el backend intente conectarse. El sistema completo se levanta desde la raíz del proyecto.

Containers [Give feedback](#)

Container CPU usage ⓘ

0.20% / 400% (4 CPUs available)

Container memory usage ⓘ

755.2MB / 3.68GB

[Show charts](#)

Q Search



☐ Only show running containers

☐	Name	Container ID	Image	Port(s)	Actions
☐	musarum_servidor	-	-	-	
☐	musarum-backend	006a2969de3e	musarum_servidor-musarum-backend	8080:8080 ↗	
☐	musarum-mariadb	eb346e826f42	mariadb:latest	3305:3306 ↗	

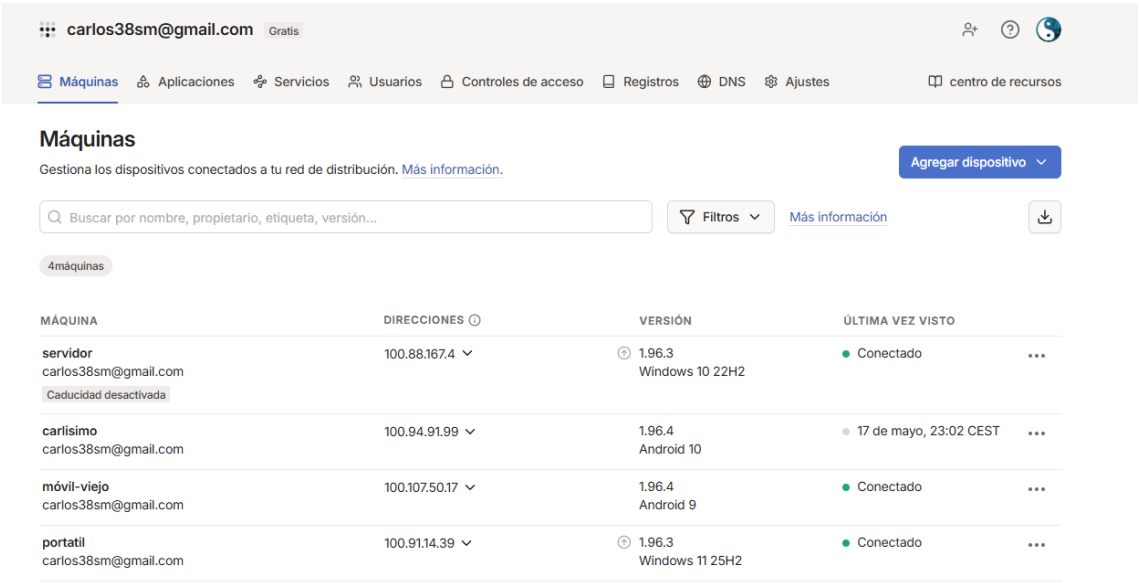


API BASE: <http://localhost:8080/api>
SWAGGER : <http://localhost:8080/api/swagger-ui/index.html>

```
12:27:11 INFO [ restartedMain] BackendApplication : Starting BackendApplication using Java 21.0.7 with PID 21908 (C:\Users\c
12:27:11 INFO [ restartedMain] BackendApplication : No active profile set, falling back to 1 default profile: "default"
12:27:16 INFO [ restartedMain] FileStorageServiceImpl : MUSARUM INICIALIZADO EN: C:\Users\carlo\musarum
12:27:17 INFO [ restartedMain] BackendApplication : Started BackendApplication in 6.149 seconds (process running for 6.809)
```

La persistencia de datos aplica dos estrategias. Los datos estructurados residen en MariaDB dentro de un volumen nombrado (*musarum_mariadb_data*). Los ficheros de audio se almacenan en el host, mapeados al contenedor mediante *volumen bind*.

El acceso remoto se resuelve con Tailscale, una red privada virtual que establece túneles cifrados entre dispositivos sin configurar cortafuegos ni redirecciones de puertos. Una vez autenticados con la misma cuenta, todos los dispositivos quedan conectados en una red virtual con direccionamiento **IP persistente**. En Musarum, esto permite que el servidor doméstico alojado en un portátil sea accesible desde cualquier ubicación. El sistema mantiene redundancia mediante tres instancias del backend: una desplegada en el servidor doméstico de Lebrija, otra ejecutándose localmente en el portátil de presentación y otra contenerizada también en el portátil de la presentación, permitiendo conmutar entre las tres ante cualquier imprevisto durante la demostración.



The screenshot shows the Tailscale web interface for a user named 'carlos38sm@gmail.com'. The interface includes a navigation bar with options like 'Máquinas', 'Aplicaciones', 'Servicios', 'Usuarios', 'Controles de acceso', 'Registros', 'DNS', and 'Ajustes'. The main section is titled 'Máquinas' and contains a table of connected devices. The table has columns for 'MÁQUINA', 'DIRECCIONES', 'VERSIÓN', and 'ÚLTIMA VEZ VISTO'. There are four machines listed: 'servidor' (Windows 10 22H2), 'carlisimo' (Android 10), 'móvil-viejo' (Android 9), and 'portatil' (Windows 11 25H2). Each machine entry includes its IP address, version, and status (e.g., 'Conectado').

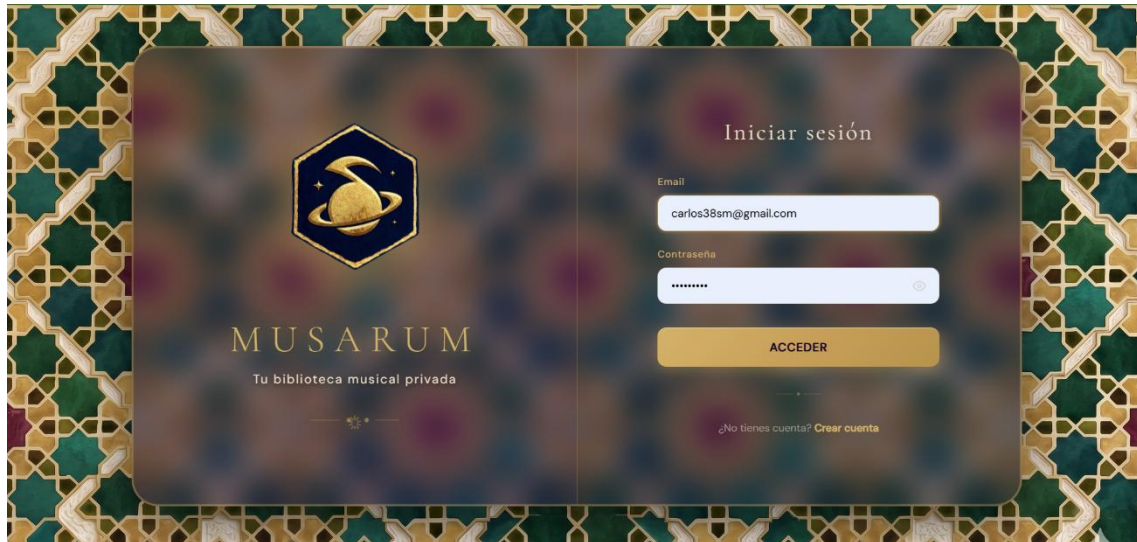
MÁQUINA	DIRECCIONES	VERSIÓN	ÚLTIMA VEZ VISTO
servidor carlos38sm@gmail.com Caducidad desactivada	100.88.167.4	1.96.3 Windows 10 22H2	Conectado
carlisimo carlos38sm@gmail.com	100.94.91.99	1.96.4 Android 10	17 de mayo, 23:02 CEST
móvil-viejo carlos38sm@gmail.com	100.107.50.17	1.96.4 Android 9	Conectado
portatil carlos38sm@gmail.com	100.91.14.39	1.96.3 Windows 11 25H2	Conectado

Este diseño combina las ventajas del despliegue contenerizado con la flexibilidad de un servidor doméstico, evitando costes recurrentes de servicios en la nube y manteniendo el control total sobre los datos personales.

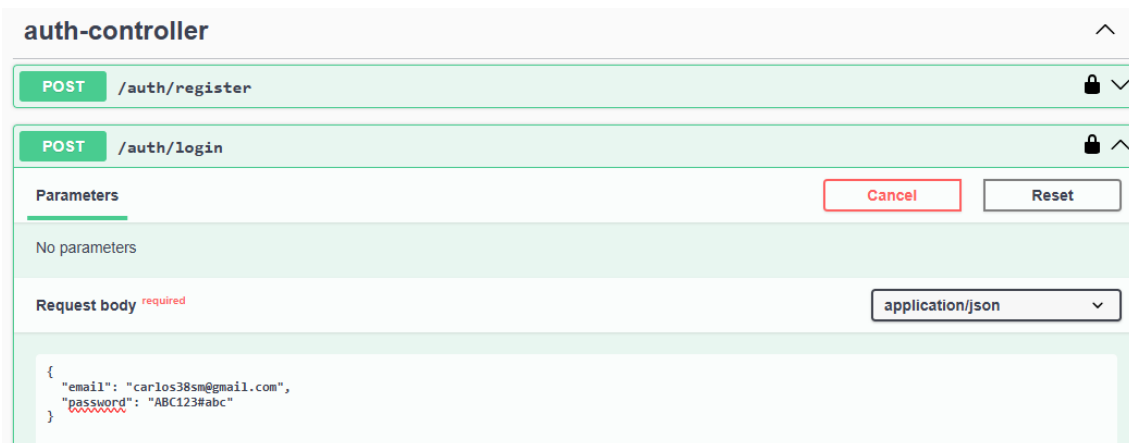
El backend sirve sobre HTTP dentro de la red privada de Tailscale. Aunque no se implementa HTTPS a nivel de aplicación, la confidencialidad del tráfico queda garantizada por el cifrado extremo a extremo que Tailscale aplica a toda comunicación entre nodos. Dado que el servidor no expone puertos a internet y solo es accesible a través de la VPN, no existe tráfico en claro fuera del túnel cifrado.

10. PRUEBAS Y RESULTADOS

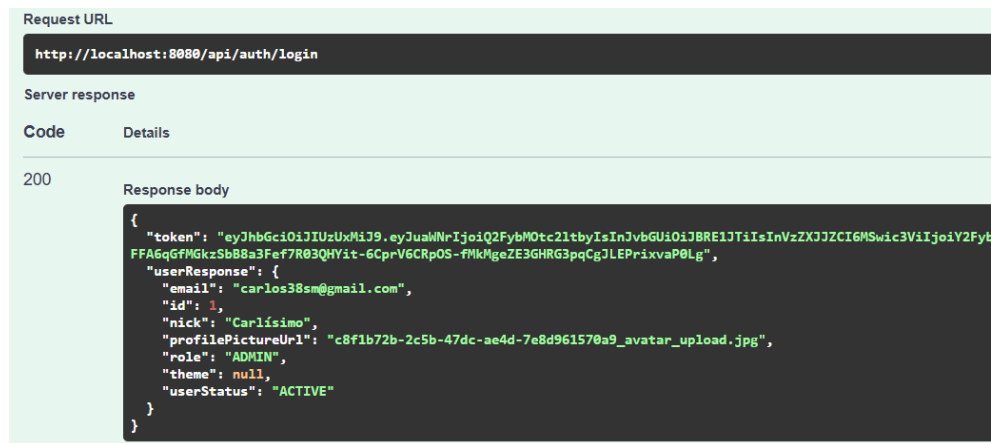
La validación se ha realizado mediante Swagger y Postman. En primer lugar, las pruebas de API con Swagger y Postman: esta interfaz permite visualizar todos los endpoints organizados por controlador, inspeccionar los DTO, y ejecutar peticiones directamente desde el navegador.



Para validar el flujo de autenticación, se ejecutó el endpoint POST /auth/login introduciendo credenciales válidas de un usuario previamente registrado. La petición se envía en formato JSON con los campos email y password.



El servidor respondió con código 200 OK y devolvió un objeto que contiene el token JWT junto con los datos del usuario. El token guarda la firma criptográfica que garantiza su integridad.



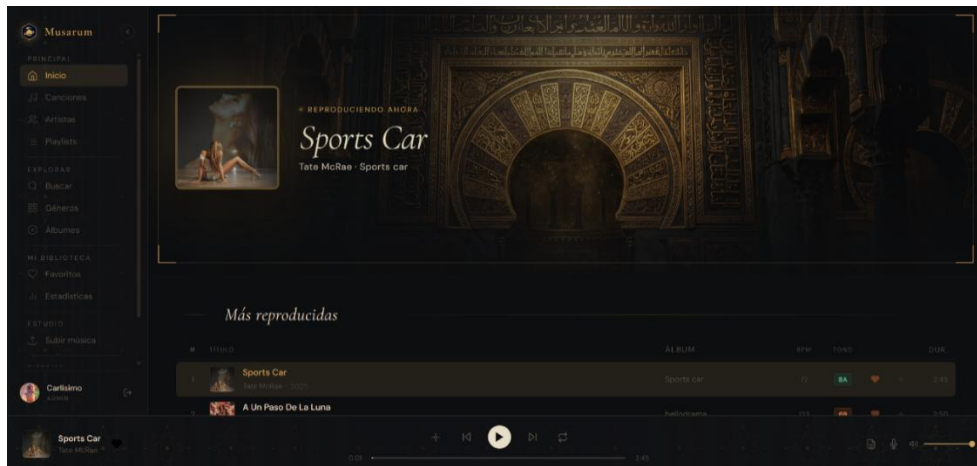
Para las peticiones siguientes que requieren autenticación, se utilizó el botón Authorize de Swagger, que abre un modal donde se introduce el token en el campo de autorización Bearer.

Con la sesión autenticada, se probó el endpoint GET /songs aplicando parámetros de paginación. La petición se construyó directamente desde la interfaz de Swagger, permitiendo ajustar los valores sin necesidad de escribir código.

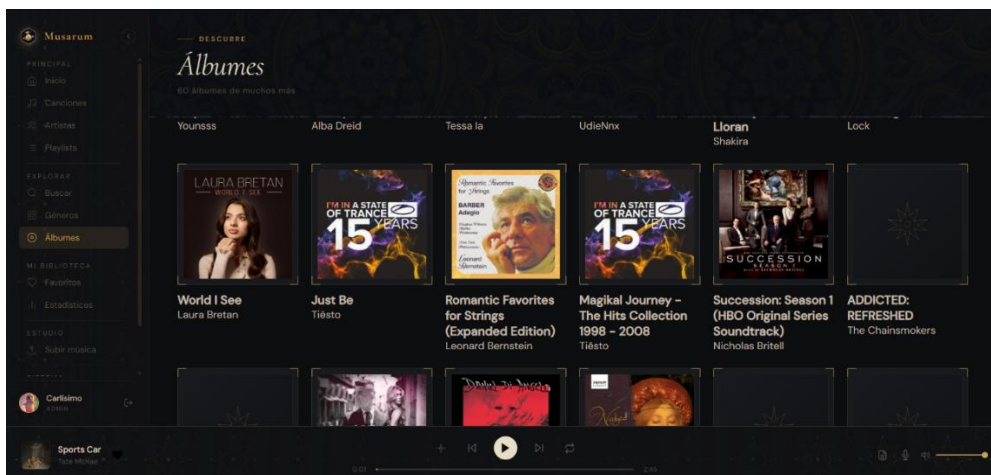


El servidor devolvió correctamente un objeto con el listado de canciones, incluyendo para cada una sus metadatos.

A continuación, la validación funcional de la web desde el cliente. Las pruebas desde Angular permitieron verificar el comportamiento del sistema en condiciones reales de uso. Tras iniciar sesión, la aplicación redirige al usuario a la pantalla principal.



La vista de álbumes muestra el catálogo completo organizado en una cuadrícula con las portadas obtenidas durante el proceso de enriquecimiento automático. La navegación entre secciones se ejecuta sin errores, confirmando que el enrutamiento del frontend y la comunicación con la API funcionan correctamente.



Los logs del backend confirman que el cliente solicita los recursos correctamente.

```
12:25:04 INFO [nio-8080-exec-2] RequestTimingFilter : OPTIONS /api/songs/filters [200] 33ms
12:25:04 INFO [nio-8080-exec-1] RequestTimingFilter : GET /api/storage/avatars/c8f1b72b-2c5b-47dc-ae4d-7e8d961570a9_avatar_upload.jpg
12:25:04 INFO [nio-8080-exec-5] RequestTimingFilter : GET /api/songs/filters [200] 389ms
12:25:06 INFO [nio-8080-exec-3] RequestTimingFilter : OPTIONS /api/songs/top [200] 1ms
12:25:06 INFO [nio-8080-exec-6] RequestTimingFilter : OPTIONS /api/albums [200] 1ms
12:25:06 INFO [nio-8080-exec-7] RequestTimingFilter : OPTIONS /api/artists [200] 1ms
12:25:06 INFO [nio-8080-exec-4] RequestTimingFilter : OPTIONS /api/songs?page=0&size=10 [200] 1ms
12:25:07 INFO [nio-8080-exec-8] RequestTimingFilter : GET /api/songs/top [200] 167ms
12:25:07 INFO [nio-8080-exec-2] RequestTimingFilter : GET /api/songs?page=0&size=10 [200] 236ms
12:25:07 INFO [nio-8080-exec-9] RequestTimingFilter : GET /api/albums [200] 460ms
12:25:08 INFO [nio-8080-exec-10] RequestTimingFilter : GET /api/artists [200] 2033ms
```

Todos los flujos se comprobaron sin errores que bloqueasen la experiencia de usuario, lo que confirma que la integración entre cliente y servidor funciona.

11. CONCLUSIONES Y MEJORAS

El desarrollo de Musarum ha sido el reto más amplio y completo al que me he enfrentado durante el ciclo formativo. Construir desde cero un ecosistema que integra un backend robusto y una aplicación web, todo ello desplegado de forma auto hospedada, ha exigido un esfuerzo continuo de aprendizaje autónomo y autogestión, especialmente al abordar tecnologías y conceptos que excedían el contenido visto durante el curso.

Durante el proyecto he comprobado la diferencia entre estudiar una tecnología de forma teórica y aplicarla en un contexto real con problemas concretos. Frameworks como Spring Boot o Angular han revelado aquí su verdadera dimensión: la necesidad de coordinar seguridad, persistencia, validación, transaccionalidad, optimización de consultas y comunicación entre capas. Trabajar con lenguajes fuertemente tipados como Java y TypeScript me ha reafirmado en mi preferencia por entornos estructurados, donde el compilador detecta errores antes de la ejecución y todo se encuentra explícito.

Este TFG me ha permitido familiarizarme con herramientas que hasta ahora conocía solo de nombre y que forman parte del flujo de trabajo profesional de cualquier desarrollador. La contenerización con Docker me ha permitido conocer cómo separar el entorno de desarrollo de la máquina anfitriona, garantizando que el sistema funcione de forma idéntica en cualquier infraestructura. El control de versiones con Git ha mantenido un histórico ordenado de cambios y permitirme revertir errores sin miedo a perder trabajo. Asimismo, Postman y Swagger han sido de utilidad para validar manualmente cada endpoint antes de integrarlo con el frontend.

A nivel arquitectónico, una de las lecciones más importantes ha sido entender cuál es la mejor estructura de organización. Comprender y decidir qué gano o pierdo en cada decisión ha sido el aprendizaje más valioso. Los conceptos abstractos cobran sentido cuando aparecen como problemas concretos.

Mejoras a futuro

Aunque la versión actual de Musarum cumple los objetivos mínimos, durante el desarrollo han surgido ideas que enriquecerían el sistema y que se han dejado documentadas como ampliaciones futuras por su complejidad:

En primer lugar, añadir un **historial de las últimas canciones reproducidas** y un sistema de **crossfade** que difumine el final con el comienzo de una canción.

En segundo lugar, una funcionalidad original que diferenciaría a Musarum sería el concepto de **diques**. Esta característica permitiría definir marcas persistentes sobre una canción y aislar el fragmento delimitado entre ambas, guardándolo como una pieza reproducible independiente.

En tercer lugar, integrar APIs que proporcionen **descripciones biográficas de artistas**, así como mostrar una visualización en tiempo real de la **transformada rápida de Fourier** (FFT) del audio, al estilo de los antiguos reproductores tipo ARES. Igualmente, podría incorporarse **Whisper** de OpenAI para generar **transcripciones automáticas de canciones** cuyas letras no estén disponibles en APIs externas.

En cuarto lugar, desarrollar un **modo coche**, una vista simplificada con botones de gran tamaño para las acciones de reproducir, pausar, avanzar y retroceder. Este modo se complementaría con un sistema de comandos por voz apoyado en la API WebSpeech del navegador, permitiendo al conductor controlar la reproducción sin apartar la mirada de la carretera.

Finalmente y quizá el punto más relevante sería la creación de una **aplicación nativa en Android** en Android Studio (IDE propio) mediante el lenguaje Kotlin (cuya base se asienta en Java) para el uso de **Musarum como Multiplataforma** real, conteniendo en sí misma tres formas de consumo: Web, Escritorio y Móvil nativo.

12. BIBLIOGRAFÍA

- Apache Software Foundation. (s.f.). Apache Maven Documentation. Recuperado el 18 de mayo de 2026, de <https://maven.apache.org/guides/index.html>
- Apple Inc. (s.f.). iTunes Search API. Recuperado el 18 de mayo de 2026, de <https://performance-partners.apple.com/search-api>
- Ben-Manes, B. (s.f.). Caffeine: A high performance caching library for Java [Repositorio de software]. GitHub. Recuperado el 18 de mayo de 2026, de <https://github.com/ben-manes/caffeine>
- DBeaver Corp. (s.f.). DBeaver Documentation. Recuperado el 18 de mayo de 2026, de <https://dbeaver.com/docs/dbeaver/>
- Deezer. (s.f.). Deezer API Documentation. Recuperado el 18 de mayo de 2026, de <https://developers.deezer.com/api>
- Docker Inc. (s.f.). Docker Documentation. Recuperado el 18 de mayo de 2026, de <https://docs.docker.com/>
- Google. (s.f.). Angular - The web development framework for building modern apps. Recuperado el 18 de mayo de 2026, de <https://angular.dev/>
- JWTK. (s.f.). JWT: Java JWT - JSON Web Token for Java and Android [Repositorio de software]. GitHub. Recuperado el 18 de mayo de 2026, de <https://github.com/jwt/jwt>
- LRCLib. (s.f.). LRCLib API Documentation. Recuperado el 18 de mayo de 2026, de <https://lrclib.net/docs>
- MariaDB Foundation. (s.f.). MariaDB Server Documentation. Recuperado el 18 de mayo de 2026, de <https://mariadb.org/documentation/>
- Microsoft. (s.f.). TypeScript Documentation. Recuperado el 18 de mayo de 2026, de <https://www.typescriptlang.org/docs/>
- MetaBrainz Foundation. (s.f.). MusicBrainz API Documentation. Recuperado el 18 de mayo de 2026, de https://musicbrainz.org/doc/MusicBrainz_API
- Mpatric. (s.f.). mp3agic: A java library for reading mp3 files and reading/manipulating the ID3 tags [Repositorio de software]. GitHub. Recuperado el 18 de mayo de 2026, de <https://github.com/mpatric/mp3agic>

Oracle. (s.f.). Java Documentation. Recuperado el 18 de mayo de 2026, de <https://docs.oracle.com/en/java/>

Postman Inc. (s.f.). Postman Learning Center. Recuperado el 18 de mayo de 2026, de <https://learning.postman.com/docs/introduction/overview/>

Project Lombok. (s.f.). Lombok Documentation. Recuperado el 18 de mayo de 2026, de <https://projectlombok.org/>

Red Hat. (s.f.). Hibernate ORM Documentation. Recuperado el 18 de mayo de 2026, de <https://hibernate.org/orm/documentation/>

SmartBear. (s.f.). Swagger Documentation. Recuperado el 18 de mayo de 2026, de <https://swagger.io/docs/>

Spring. (s.f.). Spring Boot Reference Documentation. VMware. Recuperado el 18 de mayo de 2026, de <https://docs.spring.io/spring-boot/index.html>

Tailscale Inc. (s.f.). Tailscale Documentation. Recuperado el 18 de mayo de 2026, de <https://tailscale.com/kb>

TheAudioDB. (s.f.). TheAudioDB API Guide. Recuperado el 18 de mayo de 2026, de https://www.theaudiodb.com/api_guide.php

W3C. (s.f.). HTML Living Standard. WHATWG. Recuperado el 18 de mayo de 2026, de <https://html.spec.whatwg.org/>

W3C. (s.f.). Web Speech API Specification. Recuperado el 18 de mayo de 2026, de <https://www.w3.org/TR/webspeech-api/>

W3Schools. (s.f.). W3Schools Online Web Tutorials. Recuperado el 18 de mayo de 2026, de <https://www.w3schools.com/>